# SkillPass: Technical Report

Final Version of the Product · An assessor-verified skills passport for informal artisans in Nigeria, with escrow job payments

Fejiro Obiku · BSc Software Engineering, African Leadership University · Supervisor: Tunde Isiaq Gbadamosi · October 2026 ·
Live: skillpasssss.vercel.app · Code: github.com/Fejiroobiku/skillpasssss · Video: youtube.com/watch?v=t6LdxuTkw1M

## 1. Scope alignment and feature validation

SkillPass is a full-stack web application: a React and TypeScript single-page app talks to a REST API (Node.js 22, Express 5) backed by PostgreSQL. Table 1 lists each module from the approved proposal (its Table 6) and the features added after supervisor feedback, with where each one stands now. In the app the roles are called Assessor and Artisan; the code still uses trainer and apprentice. Screenshots are in docs/screenshots, and all paths are relative to the repository root.

| Planned feature | Intended purpose | Current status | Evidence |
|---|---|---|---|
| Accounts and roles | Role-based access; assessors approved before issuing; each artisan linked to an assessor | Completed | routes/auth.js, middleware/auth.js; 13_register.jpg; tests: "authentication", "bootstrap" |
| Skill framework | Checklists per trade by level; assessors propose skills; Advanced skills need a co-signer | Completed | routes/skills.js, routes/admin.js; tests: "skills and trades" |
| Credential issuing | Assessor records an observed skill with live evidence; signed, timestamped, unique ID | Completed | routes/credentials.js, routes/evidence.js, lib/signing.js; 03_issue_credential.jpg; tests: "issuing a credential" |
| Skills passport | Shareable, printable record of verified skills | Completed | routes/passport.js (revocable link + QR); 05_apprentice_passport.jpg; tests: "public passport" |
| Public verification | Anyone checks a credential by ID, link or QR, no login | Completed | routes/verify.js; 01_verify_valid.jpg, 02_verify_revoked.jpg; tests: "public verification" |
| Integrity safeguards | Evidence first, co-signing, issuing limit, random audits, flag/revoke, trust score, append-only log | Completed | lib/scoring.js, lib/integrity.js, lib/auditChain.js; 09_review_queue.jpg, 10_audit_log.jpg; tests: "flags and disputes", "risk and misconduct" |
| Job referral and ratings | Rank verified artisans by skill and distance; employers and artisans rate each other | Partial | lib/matching.js, routes/jobs.js, routes/feedback.js; 07, 08 jpg. In-app alerts work; SMS in console mode until a Termii sender ID is approved |
| Job agreements and escrow (added 26 Oct) | Quotes split materials from workmanship; employer pays into escrow before work; release on confirmation; disputes decided by admin | Completed (test mode) | lib/escrow.js, lib/payments.js, routes/agreements.js; 14_employer_hires.jpg, 15, 16, 18 jpg; tests: agreements.test.js (18) |
| Terms of Service (added 26 Oct) | Plain-language terms in English and Pidgin, accepted at sign-up and before each job | Completed | src/data/terms.ts, lib/terms.js; 17_terms.jpg; tests: "Terms of Service" |
| Administration | Approve accounts, manage skills, review flags, audits and disputes | Completed | routes/admin.js, routes/agreements.js; 09, 11, 16 jpg; tests: "admin extras", "disputes" |
| Deployment and pilot | Hosted app; run the pilot with real users in Lagos | Partial | Live demo on Vercel + Render + Neon (README). The live pilot waits for ethics approval (submitted 23 Oct 2026) |

*Table 1. Planned features from the approved proposal and their current status.*

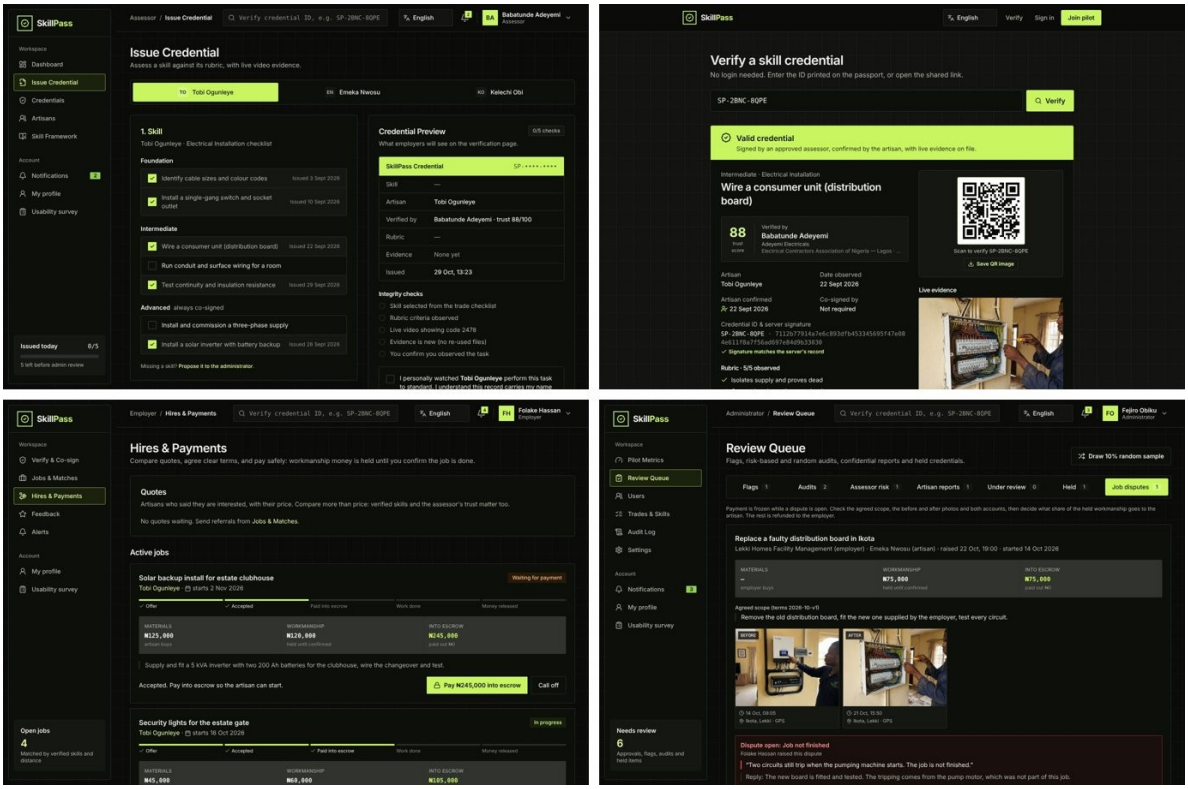



*Figure 1. Core features: an assessor issues a credential against the rubric (top left); public verification with signature check and QR code (top right); an employer's hires with money held in escrow (bottom left); an administrator deciding a job dispute from the photos and scope (bottom right).*

### Justified scope changes

**Added after supervisor feedback (26 Oct 2026).** My supervisor asked how employers and artisans would settle price, materials and payment disputes, and pointed out that the person who checks a skill is not always the one who taught it. Quotes now price materials and workmanship separately. Payment is held in escrow, the artisan uploads before and after photos, an administrator decides disputes, and artisans rate employers too. Plain-language Terms of Service cover all of it, and the roles became Assessor and Artisan.

**Added after ethics feedback.** SMS phone verification, email verification and password reset; Nigerian Pidgin; an offline upload queue; artisan-controlled passport links; a confidential report channel; consent withdrawal that removes personal data (NDPA 2023); a hash-chained audit log; and expiring signed evidence links.

**Partial or deferred.** Real SMS delivery is built (TermiiSmsChannel) but set to console mode, because Nigerian networks only deliver to Do-Not-Disturb numbers from an approved sender ID. Payments run in test mode, either simulated or through Paystack with a test key. Live money and the real pilot both wait for ethics approval. NIN integration, native apps, blockchain anchoring and NSQ integration remain out of scope.

## 2. Technical functionality

**Architecture.** The frontend (src/) is organised by role (pages/trainer, apprentice, employer, admin, public) with shared components, an auth context and a typed API client. The backend (backend/src) separates HTTP routes (one file per area) from domain logic in lib/ (scoring, matching, signing, storage, messaging, payments, escrow, audit chain, privacy). The backend runs on Render, the frontend on Vercel and the database on Neon PostgreSQL. Zod validates every request body, the server checks the JWT and role on every protected route, and writes that have to stay consistent run inside a transaction.

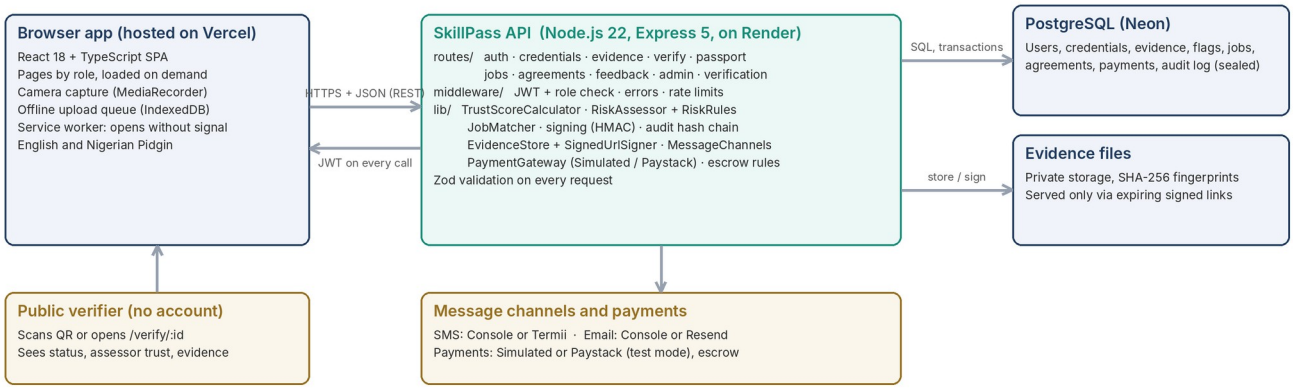



*Figure 2. System architecture: the browser app talks to the REST API over HTTPS; domain logic lives in classes under backend/src/lib.*

**Core flow: issuing and verifying a credential.** (1) The assessor gets a one-time challenge code (15 minutes). (2) The app records a live video showing it; the upload is checked by magic bytes, hashed with SHA-256 on the server and refused if that hash already backs another credential. (3) The assessor ticks the rubric and signs. The API locks the assessor's row (SELECT … FOR UPDATE) so two requests cannot both slip under the daily limit, applies probation and co-sign rules, then signs (HMAC-SHA256) and logs the credential. (4) The credential becomes valid once the artisan confirms it. (5) /verify/:id recomputes the signature and the live trust score.

**Core flow: hiring with escrow.** (1) The artisan quotes workmanship, plus materials if they buy them. (2) The employer offers an agreement with a written scope; both accept the job terms, and the Terms version is stored on the agreement. (3) The employer pays the full amount through the PaymentGateway; on confirmation the agreement moves to in progress and a materials payout is queued. (4) The artisan adds live before/after photos and marks the job done, which starts a review period (3 days). (5) The money is released when the employer confirms or when the period runs out. A dispute from either side freezes it until an administrator splits it, from 0 to 100%. The database changes, including pending payment rows, happen in one transaction, and the provider is only called after the commit, so a slow or failed provider cannot leave the data half changed.

**Testing with different inputs.** The API returns a specific error for each bad input and the UI shows it: wrong password or suspended account (401/403); an under-18 date of birth at sign-up (400, "SkillPass pilot participants must be 18 or older"); a reused challenge code (400); a duplicate evidence file (409, also written to the audit log); a credential beyond the daily limit (held for review); confirming someone else's credential (404); an unknown credential ID ("Not found" page); an edited or expired evidence link (403); a tampered credential signature ("Signature does not match"); a job offer without accepting the terms, paying before the artisan accepts, marking a job done without photos, a second open dispute, or a non-admin deciding one (400/403/409). All are covered by automated tests.

## 3. Testing strategies and results

| Strategy | Why chosen | How conducted | Result and improvement made |
|---|---|---|---|
| Unit testing | The integrity logic (trust, risk, matching, signed links) must be exactly right and is easy to test in isolation | 37 Node test-runner tests on the domain classes with fixed clocks and fake inputs, including the Paystack gateway against a fake network (kobo amounts, HMAC-SHA512 webhooks); no database | 37/37 pass. Found a time-boundary edge in the signed-link test; fixed the test clock |
| Integration (API + DB) | Integrity rules depend on SQL, transactions and locks working together | 100 tests run the Express app against a real PostgreSQL database, wiped and re-seeded per run, including the whole escrow flow, automatic release and disputes | 100/100 pass. Found a seed notice that told an assessor about a pending audit; replaced it |
| End-to-end UI | Checks that the real screens and the real API work together | 24 Vitest + Testing Library tests render the app against the running API; Playwright walks every page and the full hiring flow | 24/24 pass. Found a passport crash, phone-width overflow and a one-option trade dropdown; all fixed |
| Security | Evidence and records are sensitive; claims in the ethics application must hold | Forged, edited and expired evidence links; tampered signatures; audit-log tampering with triggers disabled; access across roles | All pass. Found that evidence was served by a permanent address; added expiring HMAC-signed links |

| Strategy | Why chosen | How conducted | Result and improvement made |
|---|---|---|---|
| Performance and compatibility | Users have low-cost Android phones on 3G/4G | Playwright + Chrome DevTools throttling (CPU and network); API load test; 4 screen sizes (Section 4) | Cut JavaScript by 44% and first-load time by 16 to 27% on a low-end phone (Section 4) |

*Table 2. Testing strategies. Totals: 161 automated tests (137 backend, 24 frontend), all passing; TypeScript and ESLint report 0 errors.*

## 4. Performance and compatibility

**Method.** The production build was served with gzip, as Vercel does, and each page was loaded with an empty cache 5 times per profile in Chromium, using Chrome DevTools Protocol throttling. Medians are reported. "Ready" is the time until the page's main content is visible.

| Profile and page | Before: ready | After: ready | JS before → after | Change in ready time |
|---|---|---|---|---|
| Low-end Android (6× CPU, 3G): sign-in | 2.69 s | 1.96 s | 243 → 135 KB | 27% faster |
| Low-end Android: employer verifies a credential | 3.23 s | 2.54 s | 243 → 135 KB | 21% faster |
| Low-end Android: artisan passport (signed in) | 4.18 s | 3.50 s | 243 → 146 KB | 16% faster |
| Mid-range Android (4× CPU, 4G): sign-in | 1.41 s | 0.91 s | 243 → 135 KB | 35% faster |
| Desktop broadband: sign-in | 0.44 s | 0.43 s | 243 → 135 KB | No real change |

*Table 3. Page performance before and after on-demand page loading (median of 5 cold loads). The gain is largest on slow phones and networks, which is what pilot users have; on a fast desktop there is almost no difference. JavaScript heap on the low-end verify page fell from 5.6 to 4.3 MB.*

**What changed and why.** The first build shipped one 574 kB JavaScript bundle. Splitting every page into its own file made the download smaller, but on 3G the content appeared later, because the browser fetched the main file, then checked the account, and only then asked for the page file. The final version keeps the public entry pages (sign-in, register, verify, passport) in the main bundle and, for a signed-in user, starts fetching their role's pages in parallel with the account check (src/pageLoaders.ts). The proposal's target was verification in under 1 minute; the verify page is ready in 2.5 s on the slowest profile.

**API under load.** I ran 10 concurrent clients against a 2-core test machine. Public verification took a median of 34 ms (p95 81 ms) at about 260 requests per second, and an artisan's full data load took 47 ms (p95 59 ms). The heaviest request, an administrator's data with live trust and risk scores for every assessor, took 147 ms (p95 192 ms). No request failed.

**Compatibility.** Every screen was checked at 360 px (low-end Android), 390 px (iPhone-size), 768 px (tablet) and 1366 px (laptop) for horizontal overflow; after fixes, none overflows (docs/screenshots/compat_*.jpg). Automated runs used Chromium, the engine behind Chrome, Edge and Opera, which are the most common Android browsers in Nigeria. Firefox and Safari were not available in the test environment, so the walkthrough video shows them. The deployed demo runs on free tiers; the API sleeps when idle, so the first request after a pause takes up to a minute. The app targets modern evergreen browsers (ES2020, CSS custom properties) and needs no plug-ins; the camera uses the standard MediaRecorder API, and a browser that cannot record gets a clear message instead of a broken screen.

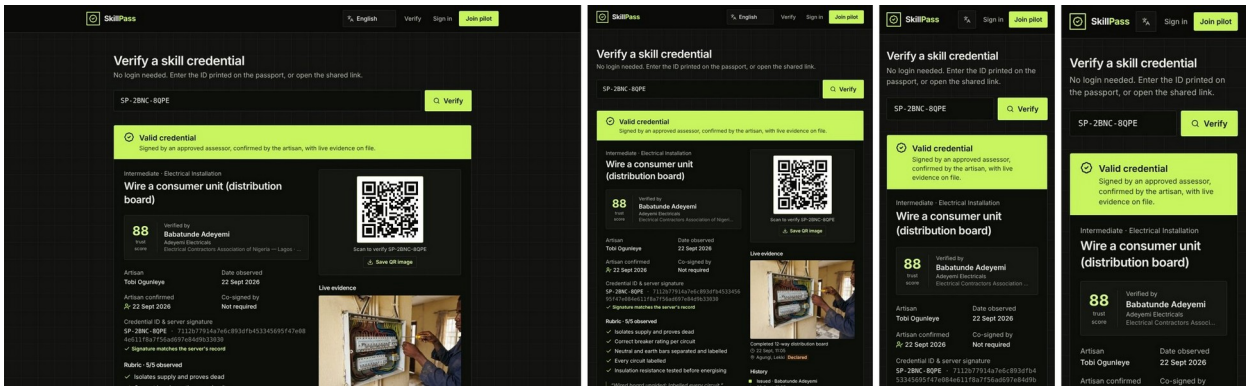



*Figure 3. The public verification page at 1366, 768, 390 and 360 px wide: no horizontal scrolling, header kept on one line.*

## 5. Code quality, OOP and algorithms

### Object-oriented design

| Class or module | OOP principle shown | Benefit |
|---|---|---|
| MessageChannel → HttpChannel → TermiiSmsChannel, ResendEmailChannel; ConsoleChannel (lib/messaging.js) | Abstraction (abstract base refuses direct use), inheritance, polymorphism: callers only call send() | Switch SMS or email provider in .env; adding Africa's Talking is one subclass. Tested with a fake fetch |
| PaymentGateway → PaystackGateway, SimulatedGateway (lib/payments.js) | Abstraction and polymorphism: routes call startCheckout(), confirm(), payout(), refund() without knowing the provider | Demo runs with no money; Paystack test mode is one setting; Flutterwave would be one subclass. Live keys are refused at start-up |
| RiskRule → FastIssuingRule, PassesEveryoneRule, ClusteredDayRule, PoorRatingsRule, OpenReportsRule; RiskAssessor (lib/scoring.js) | Strategy pattern, open/closed principle: new warning signs are added as classes without editing the assessor | Each rule has its own tests; one test injects a custom NightShiftRule to check polymorphism |
| TrustScoreCalculator (lib/scoring.js) | Encapsulation: weights validated and frozen; constants named | Weights can be tuned during the pilot without touching the formula |
| JobMatcher (lib/matching.js) | Encapsulation of matching weights and the distance cap | Configurable per trade or city; validated inputs |
| EvidenceStore → LocalDiskStore; SignedUrlSigner (lib/storage.js, lib/signedUrls.js) | Abstraction with dependency injection; private methods (#mac) hide the key | A cloud store (Supabase) can replace disk storage with one subclass; links can't be forged |
| React components and contexts (src/components, src/contexts) | Composition, separation of concerns, reusable components (PassportView, ScoreRing, EvidenceCapture) | The same passport view serves artisan, assessor, employer and public pages; job parts (AgreementSteps, MoneyBreakdown, DisputeThread) serve employer, artisan and admin |

*Table 4. Object-oriented structure. The existing function names (computeTrust, rankMatches, sendSms…) remain as thin wrappers, so no route had to change.*

### Algorithms and technical problem-solving

- **Job matching:** haversine great-circle distance plus a weighted score (70% skills held, 30% closeness within 30 km), using Set look-ups, O(n·k + n log n) for n artisans and k skills.
- **Fraud signals:** one pass over time-sorted credentials finds pairs signed minutes apart, O(n log n); a hash map counts credentials per Lagos calendar day, O(n).
- **Unbiased random audits:** a Fisher-Yates shuffle with a cryptographic random source (crypto.randomInt) picks the audit sample, so every credential has the same chance and nobody can predict the picks.
- **Tamper evidence:** each audit entry stores HMAC(previous hash, entry), forming a hash chain; verification walks the chain in O(n) and pinpoints the first broken link. Credentials and evidence links are HMAC-SHA256 signed and compared in constant time (timingSafeEqual) to resist timing attacks.
- **Escrow settlement:** held workmanship is split by percentage (artisan share rounded, remainder refunded, so no naira goes missing); overdue reviews are released in one indexed sweep; Paystack webhooks are checked with HMAC-SHA512 in constant time and confirmed with Paystack before money counts as received.
- **Data integrity under concurrency:** row-level locks and transactions enforce the daily issuing limit and probation rules even when two requests arrive at the same moment; a unique index on the evidence hash blocks reuse in O(1).
- **Weak-signal resilience:** an IndexedDB queue keeps failed uploads per user and retries them when the connection returns; each evidence link expiry is rounded to the minute so repeated loads hit the browser cache.

### Quality practices

TypeScript strict checks and ESLint pass with 0 errors. Classes use PascalCase, functions camelCase, and folders follow feature and responsibility. Every API boundary validates input; start-up checks refuse missing, reused or live payment secrets. Comments explain why. There are about 30 descriptive commits, and the README covers setup, versions and structure. Still to do: the 51 lint warnings, real SMS (sender ID pending), live payments and the pilot (ethics approval pending), and an EU-region database before real participants join.